SESSION 3 – C LAB PROGRAMS

NAME: POOJA SHREE K N

REG NO: 192572129

1. **Question:** Given an m x n grid and a ball at a starting cell, find the number of ways to move the ball out of the grid boundary in exactly N steps.

Online C Compiler GCC • C11

Q4.c

```c
#include <stdio.h>

long long dp[20][20][20];

int main() {
    int m = 2, n = 2, N = 2, start_i = 0, start_j = 0;

    dp[0][start_i][start_j] = 1;

    for (int step = 0; step < N; step++) {
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                long long ways = dp[step][i][j];
                if (!ways) continue;

                if (i == 0) dp[step + 1][i][j] += ways;
                else dp[step + 1][i - 1][j] += ways;

                if (i == m - 1) dp[step + 1][i][j] += ways;
                else dp[step + 1][i + 1][j] += ways;
```

Output / Terminal

```
$ gcc Q4.c -o Q4 && ./Q4

Number of ways = 4
```

Compilation successful • Program exited with code 0

Verified output: Number of ways = 4

Complexity: Dynamic programming; time complexity O(Nmn).

2. **Question:** You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed. All houses are arranged in a circle. Adjacent houses have security systems connected, so two adjacent houses cannot be robbed on the same night.

Online C Compiler GCC • C11

Q5.c

```c
#include <stdio.h>

int max(int a, int b) {
    return (a > b) ? a : b;
}

int robLinear(int a[], int start, int end) {
    int prev2 = 0, prev1 = 0;

    for (int i = start; i <= end; i++) {
        int current = max(prev1, prev2 + a[i]);
        prev2 = prev1;
        prev1 = current;
    }
    return prev1;
}

int main() {
    int nums[] = {1, 2, 3, 1};
    int n = 4;
```

Output / Terminal

```
$ gcc Q5.c -o Q5 && ./Q5

Maximum money = 4
Rob houses 1 and 3.
```

Compilation successful • Program exited with code 0

Verified output: Maximum money = 4 | Rob houses 1 and 3.

Complexity: Time O(n), auxiliary space O(1).

3. **Question:** You are climbing a staircase. It takes n steps to reach the top. Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Online C Compiler GCC • C11

Q6.c

```c
#include <stdio.h>

int main() {
    int n = 4;

    if (n <= 2) {
        printf("Number of ways = %d\n", n);
        return 0;
    }

    int a = 1, b = 2, c;

    for (int i = 3; i <= n; i++) {
        c = a + b;
        a = b;
        b = c;
    }

    printf("Number of ways = %d\n", b);
    return 0;
```

Output / Terminal

```
$ gcc Q6.c -o Q6 && ./Q6

Number of ways = 5
```

Compilation successful • Program exited with code 0

Verified output: Number of ways = 5

Complexity: Time O(n), auxiliary space O(1).

4Question: A robot is located at the top-left corner of an m×n grid. The robot can only move either down or right at any point in time. The robot is trying to reach the bottom-right corner of the grid. How many possible unique paths are there?

Online C Compiler GCC • C11

Q7.c

```c
#include <stdio.h>

int main() {
    int m = 7, n = 3;
    long long dp[20][20] = {0};

    for (int i = 0; i < m; i++)
        dp[i][0] = 1;

    for (int j = 0; j < n; j++)
        dp[0][j] = 1;

    for (int i = 1; i < m; i++)
        for (int j = 1; j < n; j++)
            dp[i][j] = dp[i-1][j] + dp[i][j-1];

    printf("Number of unique paths = %lld\n", dp[m-1][n-1]);
    return 0;
}
```

Output / Terminal

```
$ gcc Q7.c -o Q7 && ./Q7

Number of unique paths = 28
```

Compilation successful • Program exited with code 0

Verified output: Number of unique paths = 28

Complexity: Time O(mn), auxiliary space O(mn) in this DP implementation.